

RWorksheet_prado#1

Prado, James Harry U.

2025-10-1

```
# 1. Set up a vector named age
```

```
age <- c(34, 28, 22, 36, 27, 18, 52, 39, 42, 29, 35, 31, 27, 22, 37, 34, 19, 20, 57, 49, 50, 53, 41, 51)
age
```

```
## [1] 34 28 22 36 27 18 52 39 42 29 35 31 27 22 37 34 19 20 57 49 50 53 41 51 35
## [26] 24 33 41 37 46 25 17 37 42
```

```
# Q1a: How many data points?
```

```
length(age)
```

```
## [1] 34
```

```
# 2. Find the reciprocal of the values for age.
```

```
reciprocal_age <- 1 / age
reciprocal_age
```

```
## [1] 0.02941176 0.03571429 0.04545455 0.02777778 0.03703704 0.05555556
## [7] 0.01923077 0.02564103 0.02380952 0.03448276 0.02857143 0.03225806
## [13] 0.03703704 0.04545455 0.02702703 0.02941176 0.05263158 0.05000000
## [19] 0.01754386 0.02040816 0.02000000 0.01886792 0.02439024 0.01960784
## [25] 0.02857143 0.04166667 0.03030303 0.02439024 0.02702703 0.02173913
## [31] 0.04000000 0.05882353 0.02702703 0.02380952
```

```
# 3. Assign new_age
```

```
new_age <- c(age, 0, age)
new_age
```

```
## [1] 34 28 22 36 27 18 52 39 42 29 35 31 27 22 37 34 19 20 57 49 50 53 41 51 35
## [26] 24 33 41 37 46 25 17 37 42 0 34 28 22 36 27 18 52 39 42 29 35 31 27 22 37
## [51] 34 19 20 57 49 50 53 41 51 35 24 33 41 37 46 25 17 37 42
```

```
# 4. Sort the values for age.
```

```
sort(age)
```

```
## [1] 17 18 19 20 22 22 24 25 27 27 28 29 31 33 34 34 35 35 36 37 37 37 39 41 41
## [26] 42 42 46 49 50 51 52 53 57
```

```
# 5. Find the minimum and maximum value for age.
```

```
min_age <- min(age)
max_age <- max(age)
```

```
cat("Minimum age:", min_age, "\n")
```

```
## Minimum age: 17
```

```
cat("Maximum age:", max_age)
```

```

## Maximum age: 57
# 6. Set up a vector named data
data <- c(2.4, 2.8, 2.1, 2.5, 2.4, 2.2, 2.5, 2.3, 2.5, 2.3, 2.4, 2.7)
data

## [1] 2.4 2.8 2.1 2.5 2.4 2.2 2.5 2.3 2.5 2.3 2.4 2.7
# Q6a: How many data points?
length(data)

## [1] 12
# 7. Generates a new vector for data where you double every value
data_doubled <- data * 2
data_doubled

## [1] 4.8 5.6 4.2 5.0 4.8 4.4 5.0 4.6 5.0 4.6 4.8 5.4
# 8.1 Integers from 1 to 100
seq_1_100 <- 1:100
# To show the output, we only print the head:
head(seq_1_100)

## [1] 1 2 3 4 5 6
# 8.2 Numbers from 20 to 60
seq_20_60 <- 20:60
# To show the output, we only print the head:
head(seq_20_60)

## [1] 20 21 22 23 24 25
# 8.3 Mean of numbers from 20 to 60
mean(20:60)

## [1] 40
# 8.4 Sum of numbers from 51 to 91
sum(51:91)

## [1] 2911
# 8.5 Integers from 1 to 1,000, then find only maximum data points until 10.
# Displaying the first 10 elements of the sequence 1:1000.
(1:1000)[1:10]

## [1] 1 2 3 4 5 6 7 8 9 10
# 9. Print a vector with the integers between 1 and 100 that are not divisible by 3, 5 and 7
Filter(function(i) { all(i %% c(3, 5, 7) != 0) }, 1:100)

## [1] 1 2 4 8 11 13 16 17 19 22 23 26 29 31 32 34 37 38 41 43 44 46 47 52 53
## [26] 58 59 61 62 64 67 68 71 73 74 76 79 82 83 86 88 89 92 94 97
# 10. Generate a sequence backwards of the integers from 1 to 100
seq_backwards <- 100:1
# To show the output, we only print the head:
head(seq_backwards)

## [1] 100 99 98 97 96 95

```

```
# 11. List all the natural numbers below 25 that are multiples of 3 or 5.
multiples <- (1:24)[(1:24 %% 3 == 0) | (1:24 %% 5 == 0)]
```

```
cat("Multiples of 3 or 5 below 25:\n")
```

```
## Multiples of 3 or 5 below 25:
```

```
print(multiples)
```

```
## [1] 3 5 6 9 10 12 15 18 20 21 24
```

```
# Find the sum of these multiples
```

```
sum_multiples <- sum(multiples)
```

```
cat("\nSum of these multiples:", sum_multiples)
```

```
##
```

```
## Sum of these multiples: 143
```

```
# Q11a: Data points for Q10 and Q11
```

```
cat("Data points for Q10 (100:1):", length(100:1), "\n")
```

```
## Data points for Q10 (100:1): 100
```

```
cat("Data points for Q11 (Multiples):", length(multiples))
```

```
## Data points for Q11 (Multiples): 11
```

```
# 12. Enter this statement and check x
```

```
x <- {0 + 5}
```

```
x
```

```
## [1] 5
```

```
# 13. Set up a vector named score
```

```
score <- c(72, 86, 92, 63, 88, 89, 91, 92, 75, 75, 77)
```

```
# Find x[2] and x[3] (Assuming x is score)
```

```
cat("Second element (score[2]):", score[2], "\n")
```

```
## Second element (score[2]): 86
```

```
cat("Third element (score[3]):", score[3])
```

```
## Third element (score[3]): 92
```

```
# 14. Create a vector a
```

```
a <- c(1, 2, NA, 4, NA, 6, 7)
```

```
# Change the NA display to -999 using na.print
```

```
print(a, na.print = "-999")
```

```
## [1] 1 2 -999 4 -999 6 7
```

```
# 15. Create a vector x
```

```
x <- c(2, 3, 4)
```

```
# Check for the class (x)
```

```
class_before <- class(x)
```

```
# Change the class into foo
```

```
class(x) <- "foo"
```

```

class_after <- class(x)

cat("Original class type:", class_before, "\n")

## Original class type: numeric
cat("New class type:", class_after)

## New class type: foo

# Additional Example (Assuming user inputs "Juan" for name and "25" for age)
name <- readline(prompt="Input your name: ") # User inputs name (e.g., Juan)

## Input your name:
age <- readline(prompt="Input your age: ") # User inputs age (e.g., 25)

## Input your age:
# The following lines show the variables and the final output:
print(paste("My name is", name, "and I am", age, "years old."))

## [1] "My name is  and I am  years old."
print(R.version.string)

## [1] "R version 4.5.2 (2025-10-31)"

```